

Presented by Kendrick Koh, Samuel Seow, Schaun Ma, Argel

SC4063 GROUP 12



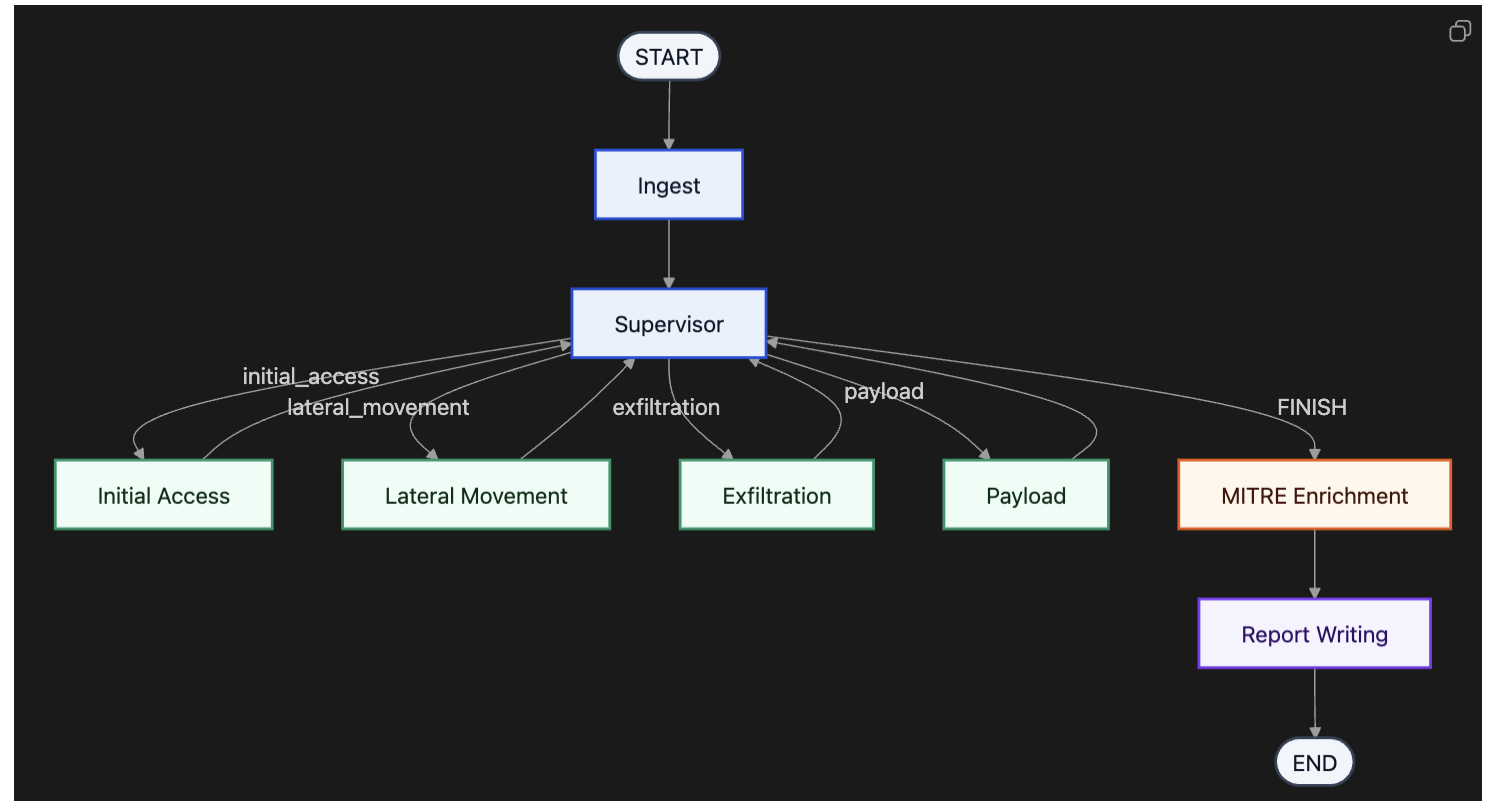
AGENT ARCHITECTURE

AGENT ARCHITECTURE

- Python
- LangGraph

- Pipeline:

Ingest -> supervisor ->
analysis agents ->
MITRE -> report



WHY LANGGRAPH?

LangGraph – Amazing for multi-step investigation rather than a single prompt-response task.

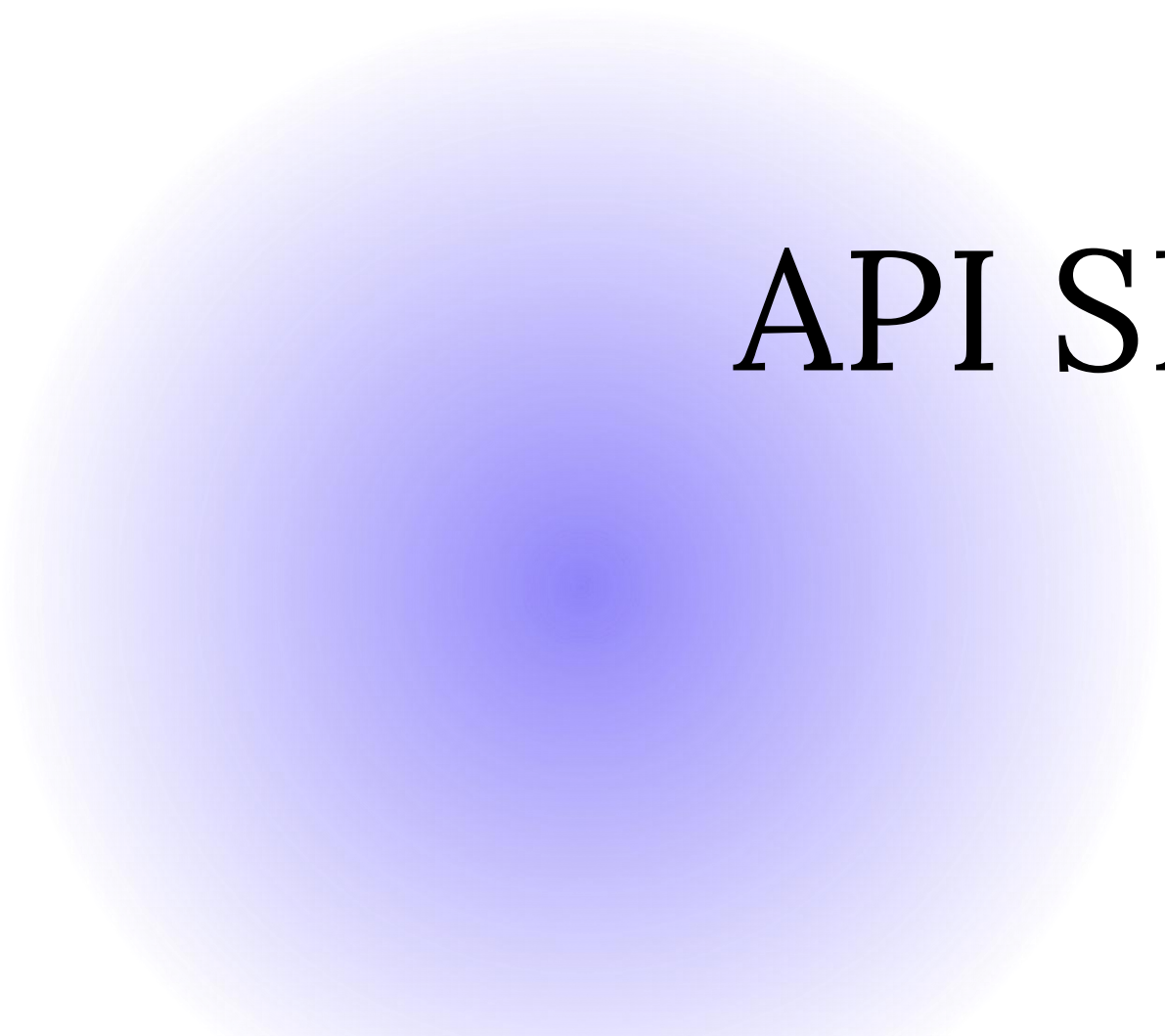
LangGraph Tools can be developed and chosen by the developers but leave tool selection to the model

Model has full control of the entire pipeline and generation and are given the tools to perform its task

```
@tool
def smb_file_drops(top_n: int = 30) -> str:
    """
    Find suspicious files written/opened via SMB: executables, scripts,
    archives, or files with ransomware-related names.
    """
```

```
@tool
def rdp_from_dc(top_n: int = 20) -> str:
    """
    Find RDP sessions originating FROM domain controllers or known
    compromised hosts to other internal hosts – indicates payload deployment.
    """
```

```
@tool
def smb_lateral_movement(top_n: int = 20) -> str:
    """
    Detect SMB-based lateral movement: file-share access, PsExec artifacts,
    remote admin share connections (ADMIN$, IPC$, C$).
    Uses Zeek SMB logs if available, otherwise falls back to tshark.
    """
```

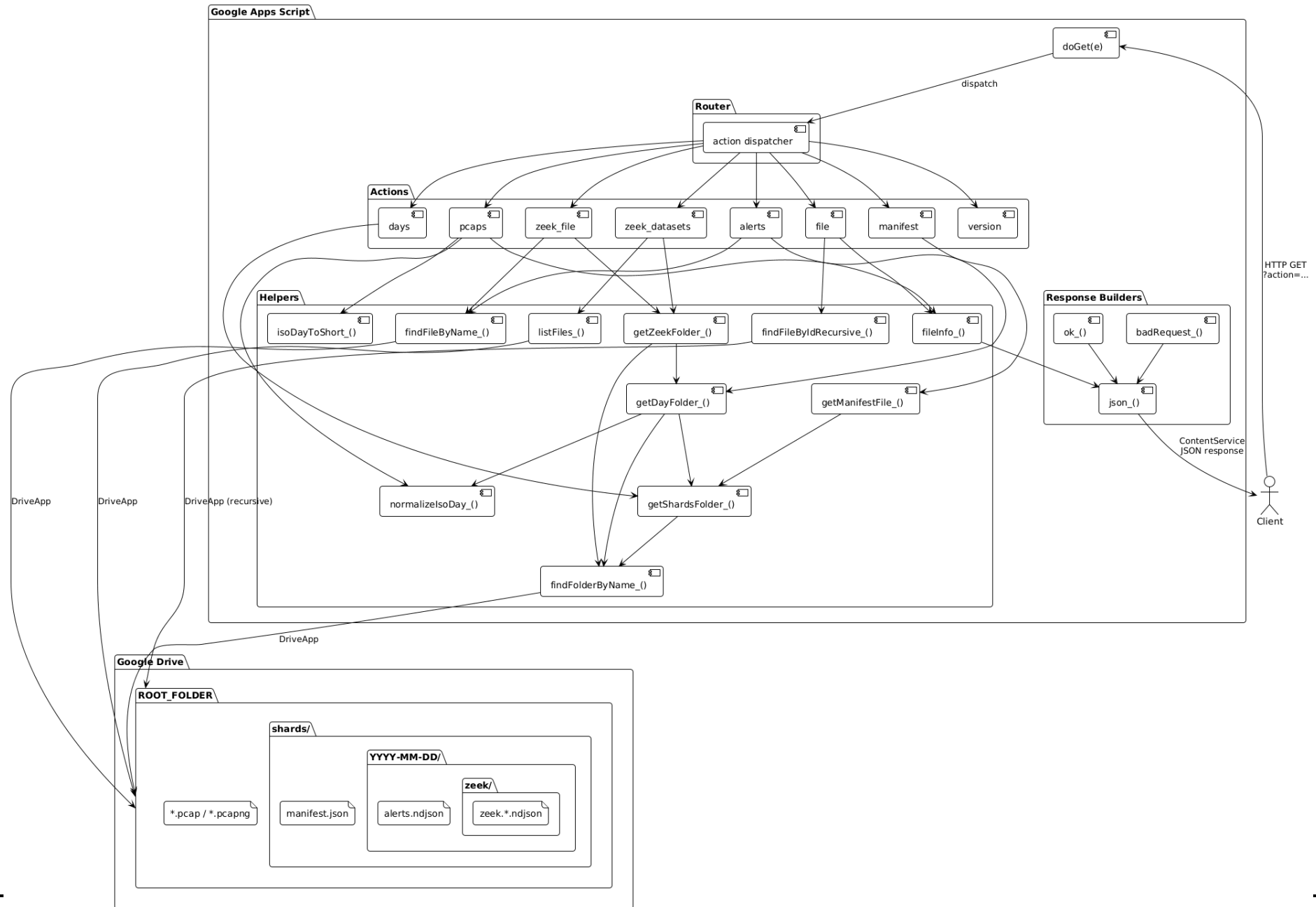


API SETUP

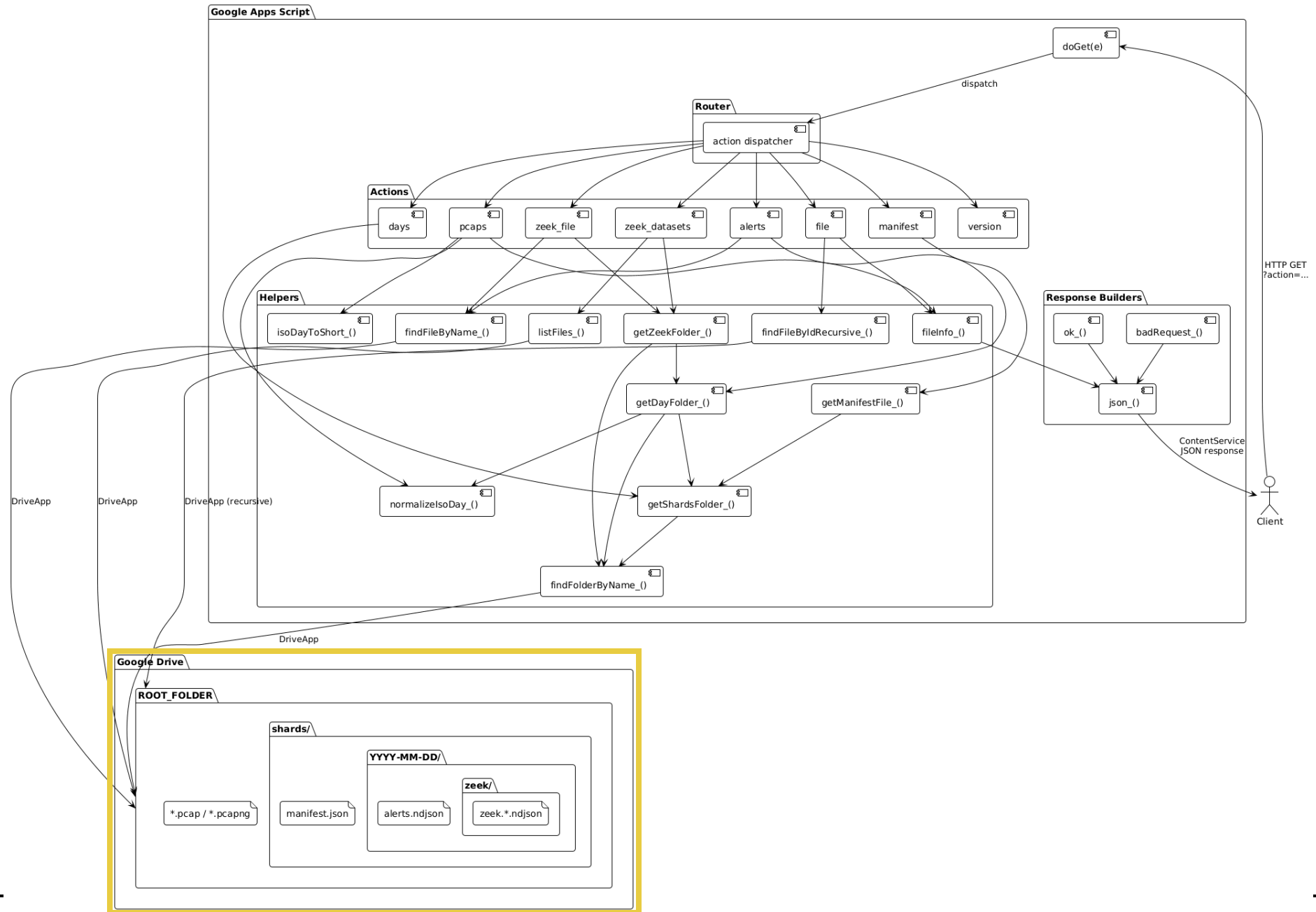
API ENDPOINT

- Combat limited disk space issue
- Single source of truth
- Break down huge Zeek and Suricata data into daily shards
- 2 tiered system
 - Look at zeek/suricata, if payload is needed, then download full PCAP

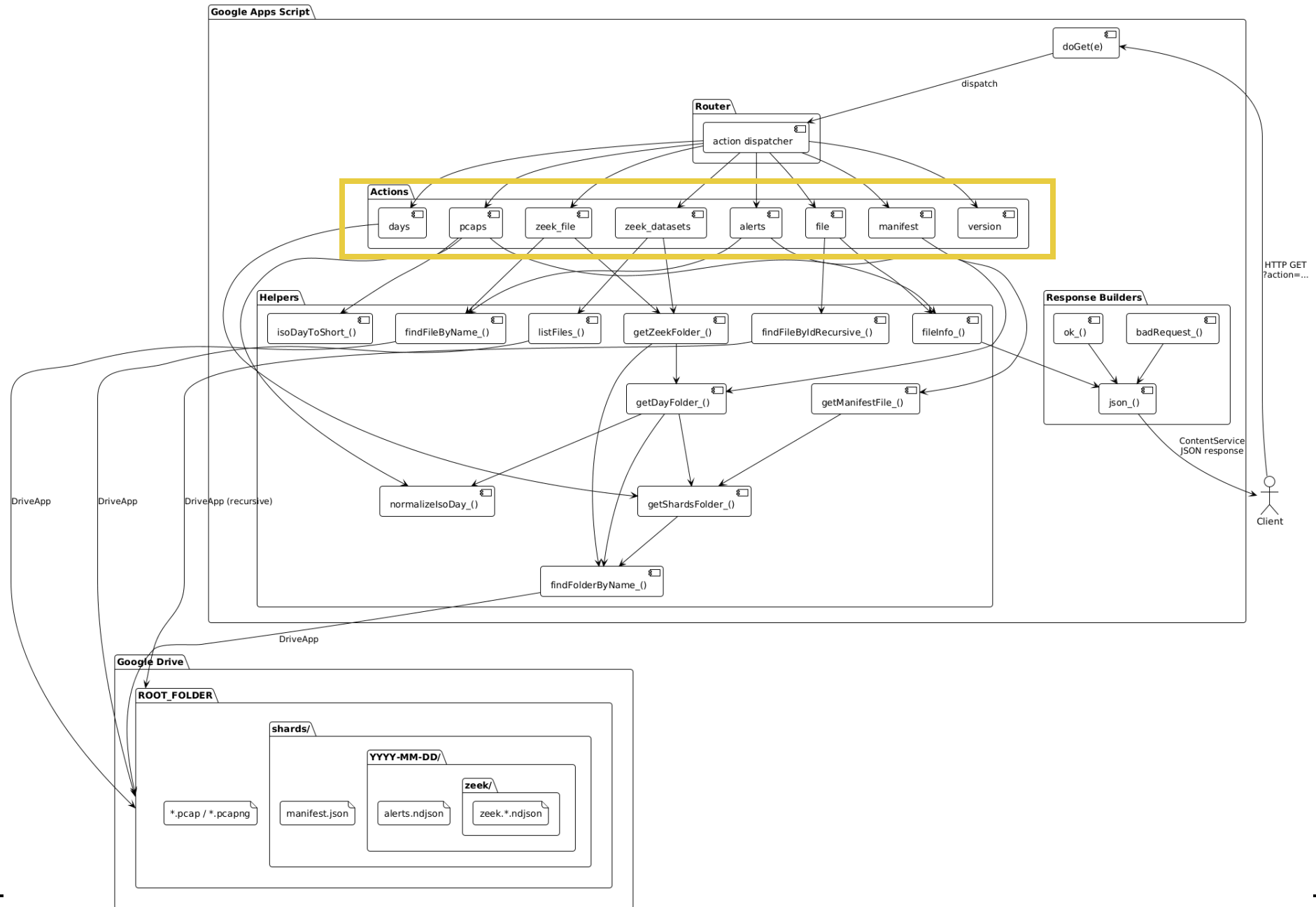
API Architecture (v8)



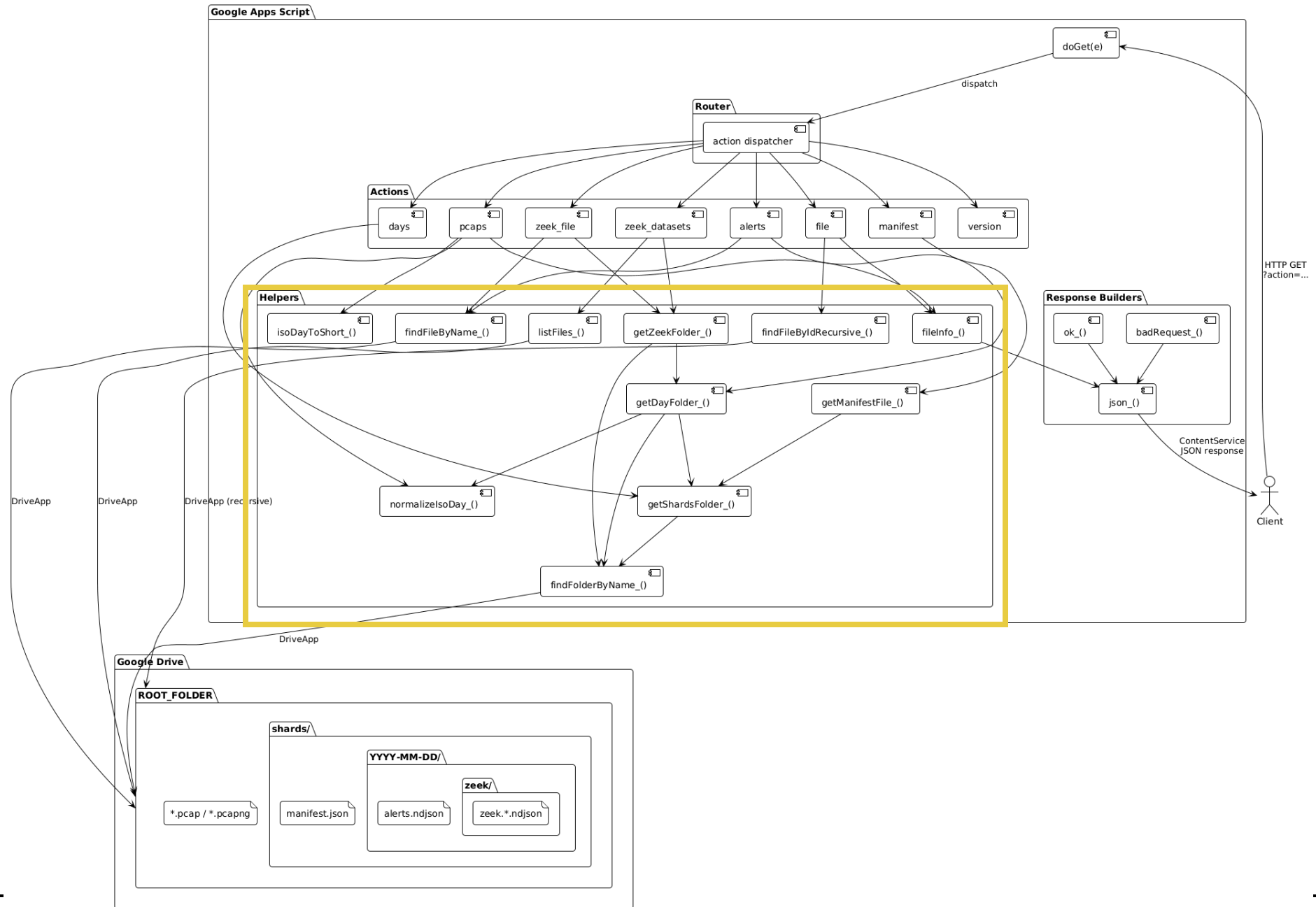
API Architecture (v8)



API Architecture (v8)



API Architecture (v8)



API ENDPOINT LIMITATIONS

- **Download method 1** Public download links – exhaust quota within minutes
- **Download method 2** Authenticated API fallback – has no quota but is much slower ~10 MB/s sustained throughput
- ~2 GB of Zeek per day x 9 days = ~18 GB downloaded at ~10 MB/s = ~30 minutes just for Zeek
- This method is fully FOC, trade off between speed and cost

API ENDPOINT – REAL WORLD SETUP

- Security Onion / Elastic Stack: Zeek logs already indexed, queried via API in milliseconds
- Local NAS: PCAPs accessed at disk speed (~500 MB/s SSD), no download needed
- S3 / Azure Blob: cloud storage with high-throughput parallel downloads (~100+ MB/s)



DATA INGESTION

RETRIEVAL LOGIC

- **Step 1: Fetch alerts**
 - Suricata alerts.ndjson
- **Step 2: Fetch Zeek datasets**
 - Zeek connection logs, DNS, RDP, HTTP, SMB file activity, etc
- **Step 3: Fetch PCAP**
 - PCAP chosen is based on later scoring logic
- **Step 4: Build `ZeekContext`**

```
@dataclass
class ZeekContext:
    """
    Pre-fetched Zeek data fed into every agent.
    Populated once by the PCAP ingestion layer, then treated as read-only.
    """
    day: str # "YYYY-MM-DD"
    pcap_path: str # local path of the downloaded PCAP
    alerts_path: Optional[str] = None # local path of alerts.ndjson
    zeek_files: dict[str, str] = field(default_factory=dict)
    # e.g. {"zeek.dns.ndjson": "/tmp/zeek.dns.ndjson", ...}
    pcap_metadata: dict[str, Any] = field(default_factory=dict)
    # raw metadata block from the API

    def to_dict(self) -> dict:
        return {
            "day": self.day,
            "pcap_path": self.pcap_path,
            "alerts_path": self.alerts_path,
            "zeek_files": self.zeek_files,
            "pcap_metadata": self.pcap_metadata,
        }

    @classmethod
    def from_dict(cls, d: dict) -> "ZeekContext":
        return cls(**d)
```

SCORING LOGIC

Score_alerts():

Analyzes the alerts and assigns scores to source IPs

- Extracts: Source IP, alert severity, rule name, hour from @timestamp
- Assigns points based on severity -> Lower severity number means more serious, and therefore more points.
- Returns a "suspect_ips" lists and "per_day" alert stats (total alerts, suspect_ip hourly density)

```
ip = a.get("source", {}).get("ip", "")
sev = a.get("event", {}).get("severity", 3)
sig = a.get("rule", {}).get("name", "")
ts = a.get("@timestamp", "")[:13] # "YYYY-MM-DDTHH"
```

```
# Score: severity 1 = 3 pts, 2 = 2 pts, 3 = 1 pt
score = max(1, 4 - sev)
ip_scores[ip] = ip_scores.get(ip, 0) + score
ip_counts[ip] += 1
```

```
for day, logs in sorted(all_logs.items()):
    alerts_path = logs.get("alerts_path")
    if not alerts_path or not Path(alerts_path).exists():
        per_day[day]["suspect_hourly"] = {}
        continue

    suspect_hourly: Counter = Counter()
    with open(alerts_path, encoding="utf-8", errors="replace") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            try:
                a = json.loads(line)
            except json.JSONDecodeError:
                continue
            ip = a.get("source", {}).get("ip", "")
            ts = a.get("@timestamp", "")[:13]
            if ip in suspect_ip_set and ts:
                suspect_hourly[ts] += 1

    per_day[day]["suspect_hourly"] = dict(sorted(suspect_hourly.items()))

return {
    "suspect_ips": suspect_ips,
    "per_day": per_day,
}
```

SCORING LOGIC

Needs_pcap_drilldown():

A day is flagged if there is evidence like a large successful RDP session, heavy brute-force activity, lateral movement, exfiltration, or simply a high alert count.

Prompts for download of specific PCAP file

```
def _needs_pcap_drilldown(day_result: dict, day_alert_count: int) -> tuple[bool, str]:
    """
    Determine if a day needs PCAP drill-down based on Zeek sweep findings.
    Errs on the side of caution - drills down if ANYTHING looks suspicious.
    """
    reasons: list[str] = []

    ia = day_result.get("initial_access_findings", {})
    lm = day_result.get("lateral_movement_findings", {})
    ex = day_result.get("exfiltration_findings", {})

    if ia.get("successful_session_bytes") and ia["successful_session_bytes"] > 10000:
        reasons.append(f"successful RDP session ({ia['successful_session_bytes']:,} bytes)")

    if ia.get("brute_force_count", 0) > 1000:
        reasons.append(f"heavy brute-force ({ia['brute_force_count']:,} attempts)")

    lm_observed = lm.get("observed", [])
    lm_hosts = lm.get("compromised_hosts", [])
    if lm_observed or lm_hosts:
        reasons.append(f"lateral movement detected ({len(lm_hosts)} hosts)")

    if ex.get("detected"):
        reasons.append("exfiltration detected")

    if day_alert_count > 50:
        reasons.append(f"{day_alert_count} alerts")

    if reasons:
        return True, "; ".join(reasons)
    return False, "no significant findings"
```


SCORING LOGIC

Score_all_pcaps():

Returns capture most aligned with the time windows that appear most suspicious.

Based on results from Score_alerts(), PCAP files are mapped back to the specific IP addresses for download

```
per_day = scoring.get("per_day", {})
all_scored: list[dict] = []

for day in sorted(per_day.keys()):
    day_info = per_day[day]
    suspect_hourly = day_info.get("suspect_hourly", {})

    pcaps = list_pcaps(day)
    if not pcaps:
        continue
    n_pcaps = len(pcaps)

    if not suspect_hourly:
        continue

    hours = sorted(suspect_hourly.keys())
    if not hours:
        continue

    # Map hours to PCAP buckets proportionally
    bucket_scores: dict[int, int] = {}
    for i, hour in enumerate(hours):
        pcap_idx = min(int(i * n_pcaps / len(hours)), n_pcaps - 1)
        bucket_scores[pcap_idx] = bucket_scores.get(pcap_idx, 0) + suspect_hourly[hour]

    for pcap_idx, pcap_score in bucket_scores.items():
        if pcap_score <= 0:
            continue
        pcap_meta = pcaps[pcap_idx]
        all_scored.append({
            "day": day,
            "pcap_index": pcap_idx,
            "pcap_name": pcap_meta.get("name", ""),
            "score": pcap_score,
            "size_mb": round(pcap_meta.get("size", 0) / 1e6, 1),
        })
```

MEMORY ALLOCATION

Disk_manager.py:

All PCAP and Zeek files downloaded are stored locally -> Leads to memory issues (breaks the entire flow)

Soft limit: 5 GB, Hard limit: 8 GB

FIFO sliding window of PCAP and Zeek files

- Old files that have been used for context are removed

```
class DiskBudgetManager:
    """
    Tracks downloaded pipeline artifacts, enforces disk budget via LRU eviction.

    Files go through: active -> released -> evicted (deleted from disk)
    Only *released* files are candidates for eviction. Active files are
    never deleted - the pipeline must explicitly release them first.
    """

    def __init__(
        self,
        work_dir: str,
        soft_limit: int = SOFT_LIMIT,
        hard_limit: int = HARD_LIMIT,
    ):
        self.work_dir = Path(work_dir).resolve()
        self.soft_limit = soft_limit
        self.hard_limit = hard_limit
        self._manifest_path = self.work_dir / ".disk_manifest.json"
        self._files: Dict[str, TrackedFile] = {}
        self._load()
```

PipelineState class:

Shared memory object for the whole LangGraph. Every node reads from it and writes back into it, so it acts like the single canonical “case file” for one run of the pipeline.

```
class PipelineState(TypedDict, total=False):
    """
    Canonical state object for the SC4063 Security Analysis Pipeline.

    ALL agents read from and write to this structure.
    Use helper functions below to safely read/write each section.
    """
    # — Identity —
    run_id: str # UUID for this pipeline run
    started_at: str # ISO-8601 timestamp

    # — Input —
    target_day: str # "YYYY-MM-DD" – the day to investigate
    pcap_file: str # resolved local path to the PCAP
    pcap_files: list[str] # all selected PCAPs for the day
    work_dir: str # scratch directory for downloads

    # — Alert scoring (populated by two-phase ingestion) —
    alert_scoring: dict[str, Any] # output of score_alerts()

    # — Zeek / API layer —
    zeek_context: dict[str, Any] # ZeekContext.to_dict() serialised

    # — Shared attack context (accumulated across agents) —
    attack_context: dict[str, Any] # rolling shared facts
    # Keys agents should write: "patient_zero", "compromised_hosts",
    # "attacker_ips", "timeline_events" (list[{"time", "event", "source"}])

    # — Per-agent findings (serialised dataclasses) —
    initial_access_findings: dict[str, Any]
    lateral_movement_findings: dict[str, Any]
    exfiltration_findings: dict[str, Any]
    payload_findings: dict[str, Any]

    # — MITRE ATT&CK enrichment —
    mitre_enrichment: dict[str, Any]
    # Keys: "matched_techniques", "candidate_threat_groups", "technique_ids"

    # — Report output —
    final_report: str

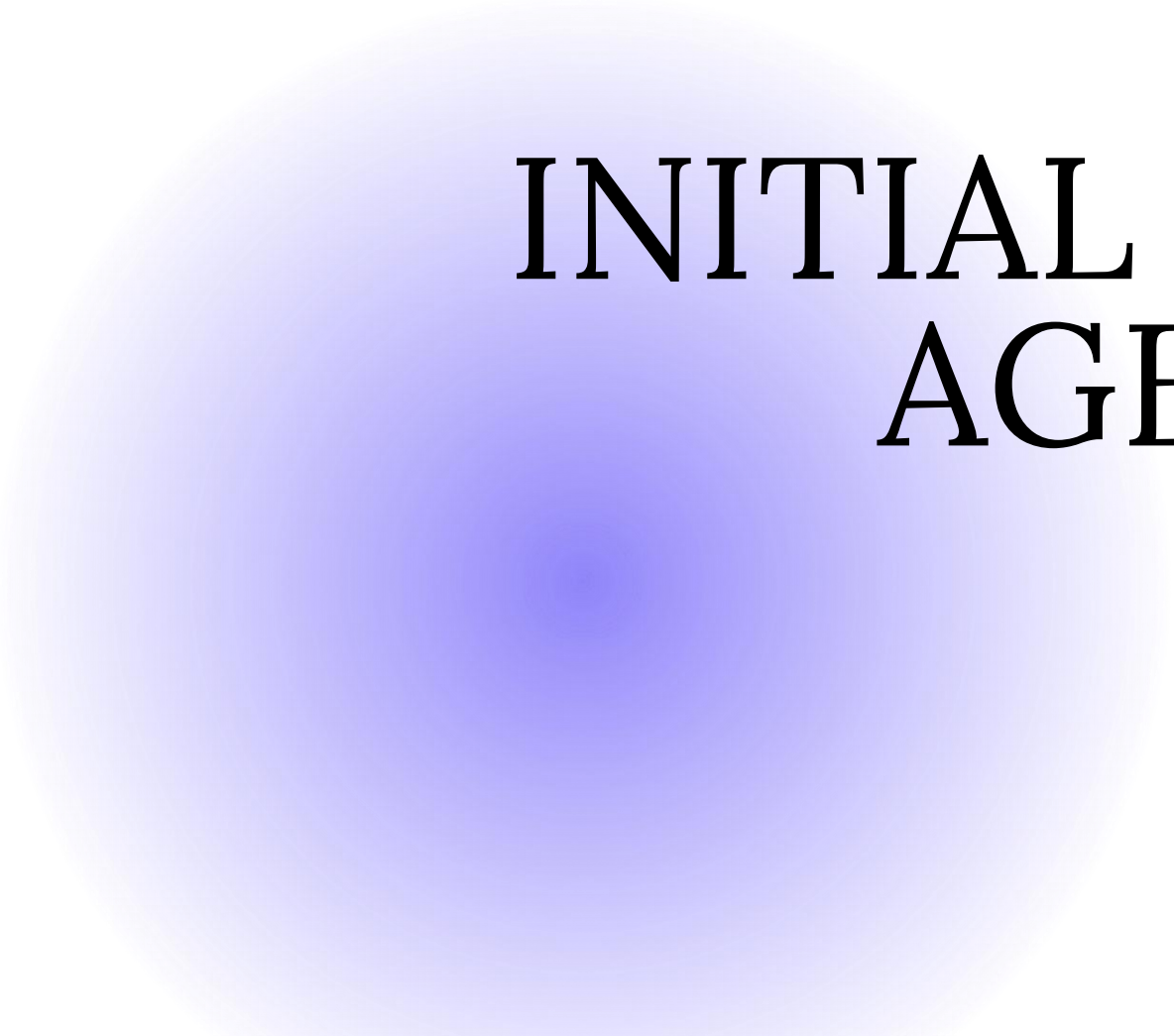
    # — Supervisor routing —
    next_agent: str
    completed_agents: list[str]

    # — LangGraph messages (append-only via add_messages) —
    messages: Annotated[Sequence[BaseMessage], add_messages]
```

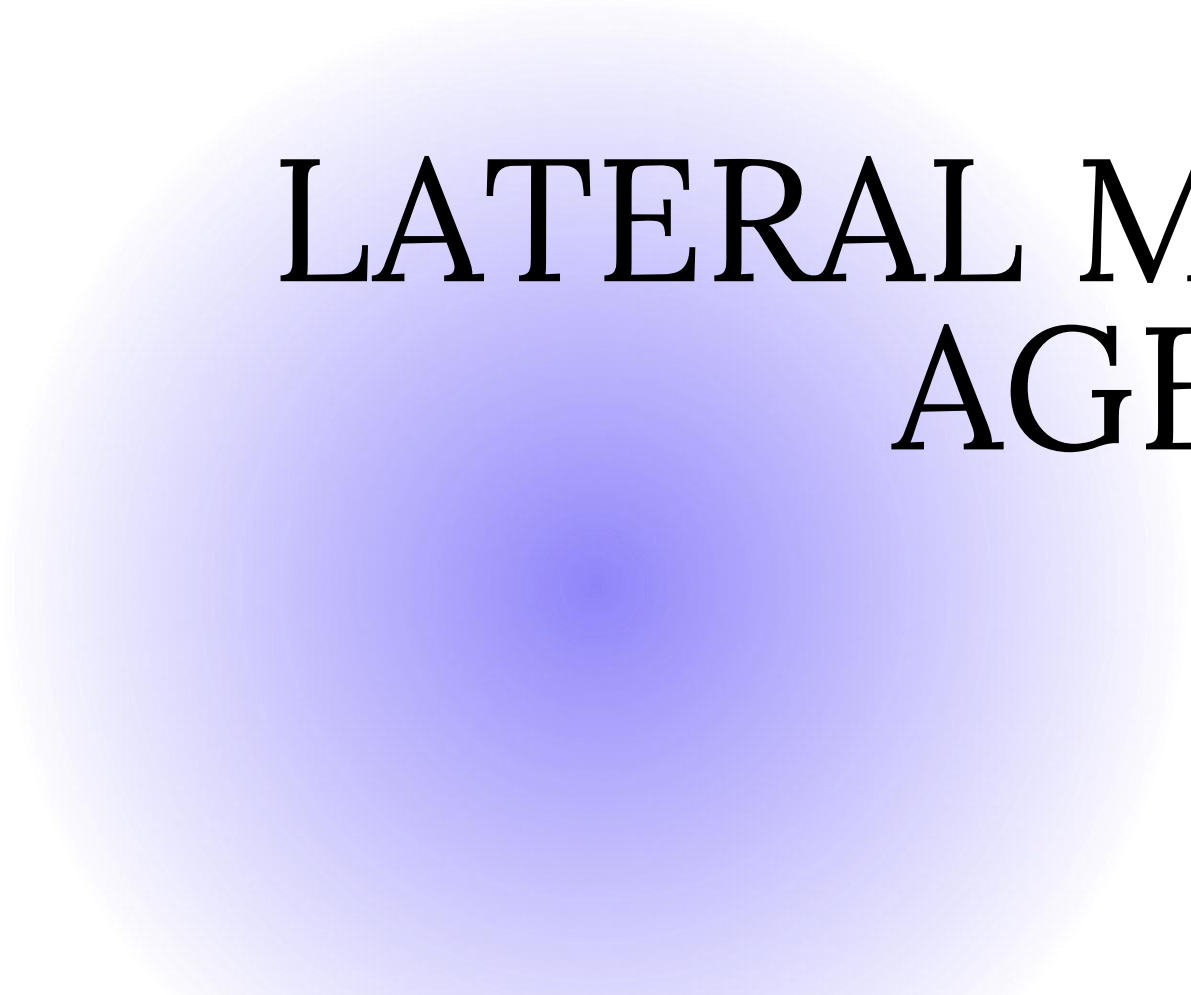
TOOLS

Available Tools

Tool	Description
<code>pcap_overview</code>	Capture statistics (time range, packets, size)
<code>protocol_hierarchy</code>	Protocol breakdown with frame counts
<code>ip_conversations</code>	Top IP conversations by bytes
<code>tcp_conversations</code>	TCP session statistics with optional filter
<code>packet_query</code>	Flexible tshark query with display filter + field extraction
<code>group_count</code>	Count packets grouped by a field (e.g., SYNs per source IP)
<code>submit_report</code>	Deliver the final Markdown report



INITIAL ACCESS AGENT



LATERAL MOVEMENT AGENT

LATERAL MOVEMENT AGENT

What it does

Detects attacker movement across internal hosts using 5 Zeek-based tools in an LLM ReAct loop.

5 Tools:

Tool	Zeek Source	What It Detects
smb_lateral_movement	zeek.smb_mapping.ndjson, zeek.smb_files.ndjson	Admin share (IPC\$) access, PsExec, file staging
rdp_lateral_movement	zeek.rdp.ndjson	Internal RDP sessions between compromised hosts
ntlm_auth_events	zeek.ntlm.ndjson (tshark fallback)	Pass-the-hash, credential relay
kerberos_events	zeek.kerberos.ndjson	Kerberoasting, AS-REP roasting, pass-the-ticket
dce_rpc_events	zeek.dce_rpc.ndjson	Remote service creation, WMI, SAMR enumeration

LATERAL MOVEMENT INPUT/OUTPUT

INPUT

Zeek logs (daily)

zeek.smb_mapping.ndjson
zeek.smb_files.ndjson
zeek.rdp.ndjson
zeek.dce_rpc.ndjson
zeek.kerberos.ndjson

Attack context (prior days)

patient_zero
attacker_ips
compromised_hosts
Accumulates across 9 days

LLM ReAct Loop

GPT-4o-mini
Temperature: 0
Max 15 steps/day
5 tools available
Deterministic output

OUTPUT

Structured findings (JSON)

- Compromised host list
- Per-technique evidence
- Confidence level (H/M/L)

MITRE ATT&CK mapping

T1021.002 • T1021.003 • T1087
Matched against 835 techniques

IOCs + provenance

IPs, timestamps, Zeek file refs
Traceable to source per day

Output feeds forward — findings from Day N injected into Day N+1 context. Attack picture compounds over 9 days.

LATERAL MOVEMENT KEY FINDINGS

6

hosts
compromised

21

DCOM activations

9

days tracked

HIGH confidence

SMB • DCOM • SAMR

WHAT WAS DETECTED

SMB Admin Shares

T1021.002

.57 → .23, .20 via IPC\$

File staging to DCs
and internal hosts

HIGH

DCOM Remote Exec

T1021.003

IRemoteSCMAActivator
targeting .32 from
multiple hosts

HIGH

SAMR Enumeration

T1087

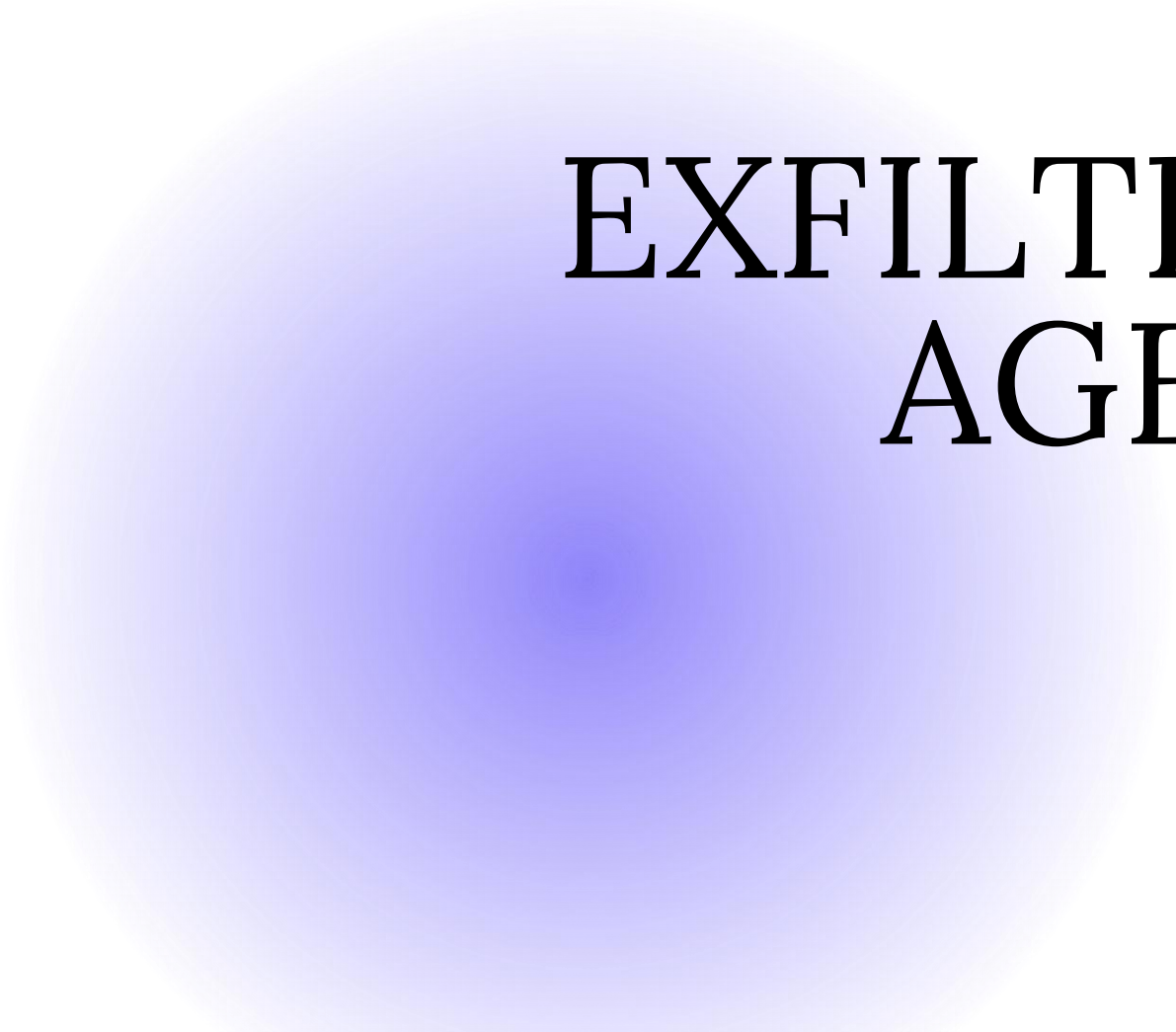
User/group enum via DCE-RPC
from .140, .79 → .20
and other hosts

HIGH

COMPROMISED HOSTS (6 confirmed)

Patient zero: 10.128.239.57 • 10.128.239.20 • 10.128.239.21 • 10.128.239.23 • 10.128.239.24 • 10.128.239.29

Detection gap: NTLM log absent all 9 days — pass-the-hash detection degraded to tshark fallback. Kerberos anomalies inconclusive due to normal AD traffic volume.



EXFILTRATION AGENT

EXFILTRATION DETECTION MODULE

Goal

To detect whether compromised internal hosts are exfiltrating data outside the network.

Input & Detection

- Input: pre-computed Zeek logs via API
- Three detection channels: general outbound, DNS tunneling, HTTP/web upload

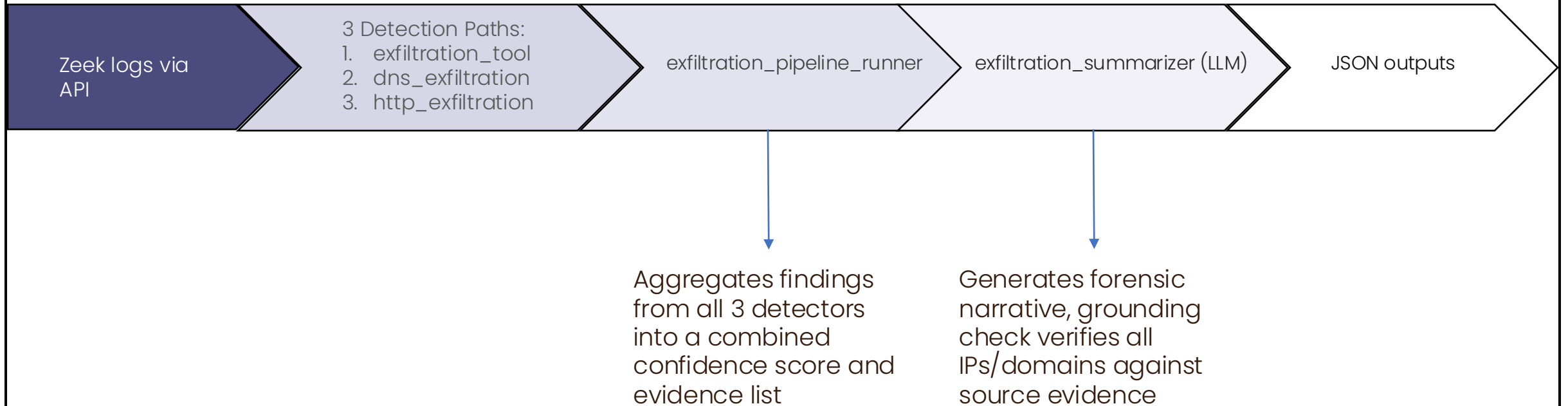
Output

- Structured JSON evidence, confidence level (LOW/MEDIUM/HIGH), MITRE ATT&CK mapping
- Optional LLM forensic summary with hallucination guard

Integration

- Agent-ready: runs as a LangGraph node, called deterministically by the master supervisor
- Tested with 30+ unit tests covering beaconing, confidence scoring, and MITRE mapping

PIPELINE ARCHITECTURE



Pipeline runs as a **LangGraph node**, called deterministically by the master supervisor

THREE DETECTION PATHS

1

General Outbound

outbound byte spikes,
unusual external
destinations, suspicious
ports/protocols,
beaconing patterns

2

DNS Tunneling

long subdomains, high-
entropy encoded labels,
elevated query rate,
suspicious query types

3

HTTP/web upload

POST/PUT/PATCH methods
with large bodies,
archive/file upload
indicators, known file-
sharing services

*Allowlisting supported across all three
trusted domains/IPs are excluded to reduce false
positives*

LLM FORENSIC SUMMARY WITH HALLUCINATION GUARD

What it produces

Executive summary, risk assessment, forensic narrative

Grounding check

Every IP/domain in LLM response verified against source evidence, grounding_score (0.0 – 1.0) and ungrounded_claims returned

System prompt constraint

Model restricted to evidence-only claims, no speculation

Other guardrails

Token budget capped, supports Azure OpenAI

OUTPUT & THREAT MAPPING

Output Fields

- Suspected: whether exfiltration is detected
- Confidence: LOW / MEDIUM / HIGH — aggregated across all three detectors
- Top internal senders and top external destinations
- Per-evidence entries: timestamp, IP pairs, bytes out, port, protocol, risk tags, domain hint

MITRE ATT&CK Mapping

- T1048 — Exfiltration Over Alternative Protocol
- T1048.003 — Exfiltration Over Unencrypted Non-C2 Protocol
- T1567 / T1567.002 — Exfiltration Over Web Service / Cloud Storage
- T1020 — Automated Exfiltration
- T1071.004 — Application Layer Protocol: DNS

Results on this dataset

- No exfiltration detected across all 9 days (suspected: False, confidence: LOW)
- Likely cause: temp.sh traffic used HTTPS — invisible to Zeek's HTTP log without TLS inspection



PAYLOAD AGENT

PAYLOAD AGENT

The **payload agent** is focused on indicators such as:

- Suspicious SMB file drops
- RDP sessions from compromised hosts or domain controllers
- WinRM / WMI / service creation for remote execution
- Suspicious HTTP payload downloads
- Archive or executable artifacts like .7z, .zip, .exe, .dll, .ps1

PAYLOAD TOOLS

- `Smb_file_drops()`
 - Looks in related `zeek.smb_files.ndjson` for suspicious filenames and extensions. (.exe, .dll, .ps1, .7z, .zip)
- `Rdp_from_dc()`
 - Looks for internal RDP sessions from Zeek connection logs, DNS, RDP, HTTP, SMB file activity, etc
- `Remote_execution_events()`
 - Looks for remote execution patterns in DCE-RPC and HTTP logs, looks for signals for: WMI, WinRM, etc
- `Http_payload_downloads()`
 - Looks for suspicious HTTP downloads
 - `extract_http_objects()`
 - `extract_smb_objects()`
- `Inspect_file()`
 - SHA256 hash, magic number, file type, entropy

PAYLOAD OUTPUT

Output Fields

Summary, deployment_method, payload_files, source_host, target_hosts, techniques, evidence_highlights, report_markdown

MITRE ATT&CK Mapping

- T1021.002 — SMB/Windows Admin Shares
- T1021.003 — Distributed Component Object Model
- T1021.001 — Remote Services (DCOM, WMI)
- T1047— Windows Management Instrumentation

Results from this dataset

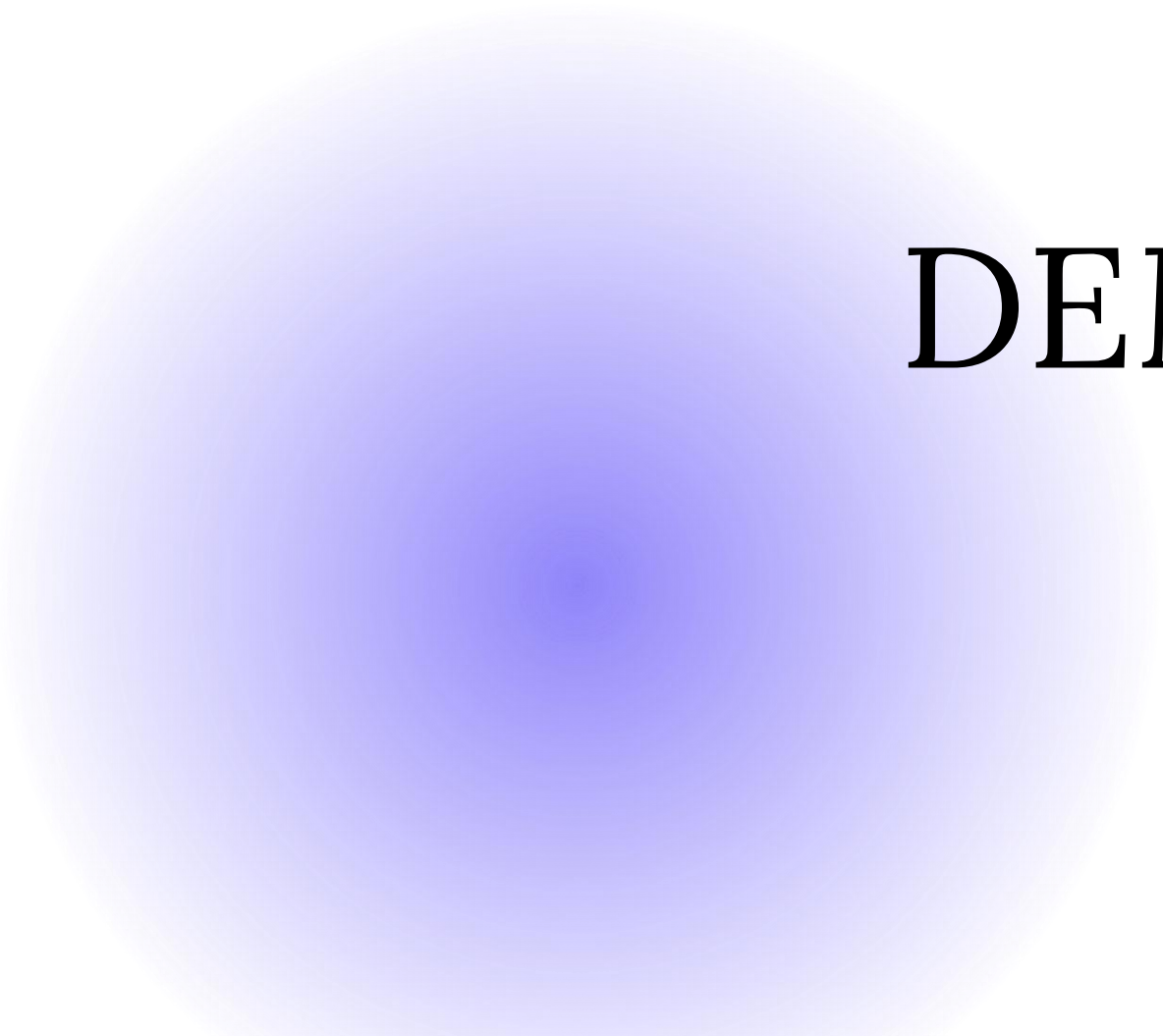
- MB file drop of UninstallWinClient.exe from 10.128.239.57 to 10.128.239.66 (2025-03-08T09:06:19.788Z)
- SMB file drops of multiple executables and archives to hosts 10.128.239.34, 10.128.239.176, 10.128.239.36, 10.128.239.32
- 21 DCOM remote activation events targeting 10.128.239.32 from multiple compromised hosts
- WMI remote login and query events from 10.128.239.82 to 10.128.239.20



REPORT WRITING AGENT

REPORT WRITING AGENT

- Report Writer
 - LLM generates structured incident report from all findings
- Combined Report Generation
 - LLM synthesises all 9 days into single MD + PDF
- Output Layer:
 - LLM combines all per-day findings into a single incident report (MD + PDF)
 - Evidence provenance appendix records exactly which files each agent accessed



DEMO



KEY CHALLENGES

TECHNICAL CHALLENGES

- Smaller local models such as Qwen 1.5B or 7B are not capable enough for this topic.
 - Explore more powerful models like GPT-4o
- Tools developed expected standard Zeek JSON, format mismatch since API outputs ECS format
 - Added normalisation layer

FALSE POSITIVES OR MISSED DETECTIONS

- SMB
 - (Lateral Movement) Normal AD operations (GPO updates, DC replication) mimic SMB lateral movement patterns
 - (Lateral Movement) Agent filters $\text{src} \neq \text{dst}$ on internal-to-internal connections to reduce noise

TOOL LIMITATIONS

- (Lateral Movement) Kerberos anomalies inconclusive, normal AD Kerberos volume masks malicious tickets
- (Exfiltration) The exfiltration agent found no evidence across all 9 days, even though the project brief stated Lynx used temp.sh for data exfiltration via large HTTP POST transfers. Possible causes include: exfiltration may use HTTPS (invisible to Zeek HTTP logs without TLS inspection), data may have been compressed/encoded before upload, and HTTP POST detection thresholds may be too conservative.

SCALING OR PERFORMANCE ISSUES

- The API serving logs and PCAPs is currently bound to Google Drive's sizing issues. For analysis that requires more logs or PCAPs, alternative storage arrangements have to be put in place
- The current setup is able to be run locally. However if more moving parts are introduced (e.g., more APIs, tools running in parallel), it would be a good idea to explore cloud infrastructure
- Currently, tools pass information to one another. More complex analysis might require multiple PCAPs and compacted context to be passed back and forth. To prevent latency issues, a cache can be considered for future development.



GUARDRAILS & SAFETY CONTROLS

Strong system prompts

Example: Report writing agent's system prompt

- Every significant claim must include supporting evidence references
- Prefer concrete references like 5-tuples, host/user IDs, timestamps, and log sources
- If evidence is missing, write "Insufficient evidence" rather than guessing
- Never fabricate packet IDs, hashes, users, hosts, ATT&CK IDs, or timestamps

```
_REPORT_SYSTEM_PROMPT = """\
You are a senior incident response report writer for an SC4063-style network forensics case.

Core requirements:
- Produce a complete report with the exact sections below in this order:
  1) Title/Cover Page
  2) Table of Contents
  3) Executive Summary (C-suite audience: root cause, business impact, recommendations)
  4) Detailed Findings
  5) Conclusion and Recommendations (prioritized High/Medium/Low)
  6) Appendix - Timeline
  7) Appendix - Additional Technical Details
  8) Evidence Gaps

Detailed Findings requirements:
- Map observations to MITRE ATT&CK tactics, techniques, and sub-techniques.
- Use the provided MITRE enrichment data for accurate technique IDs and names.
- Cover at minimum: Initial Access, Lateral Movement and Discovery, Exfiltration, Payload.
- For each candidate threat group, assess likelihood based on technique overlap.
- Explicitly identify assumptions, confidence level, and scope limits.
- Include tools used by adversary and analyst where evidence supports it.

Evidence and anti-hallucination rules:
- Every significant claim must include supporting evidence references.
- Prefer concrete references: 5-tuples, host/user identifiers, timestamps, log sources.
- If evidence is missing, write "Insufficient evidence" rather than guessing.
- Never fabricate packet IDs, hashes, users, hosts, ATT&CK IDs, or timestamps.

Exfiltration specifics to check when present:
- Outbound spikes to file-sharing services (especially temp.sh)
- Large HTTP POST transfers
- Compression indicators (e.g., 7-Zip magic bytes)

Initial access and movement specifics to check when present:
- External source into RDP/VPN followed by changed traffic behavior
- Noisy scans over SMB/RPC (ports 445/135)
- DCERPC patterns suggesting user/group modifications
- Potential RDP-based payload drop after domain controller control

Writing style:
- Professional and concise. Use markdown headings and tables when useful.
- Separate facts from analyst interpretation.
- Keep recommendations actionable and prioritized.
```

HUMAN-IN-THE-LOOP

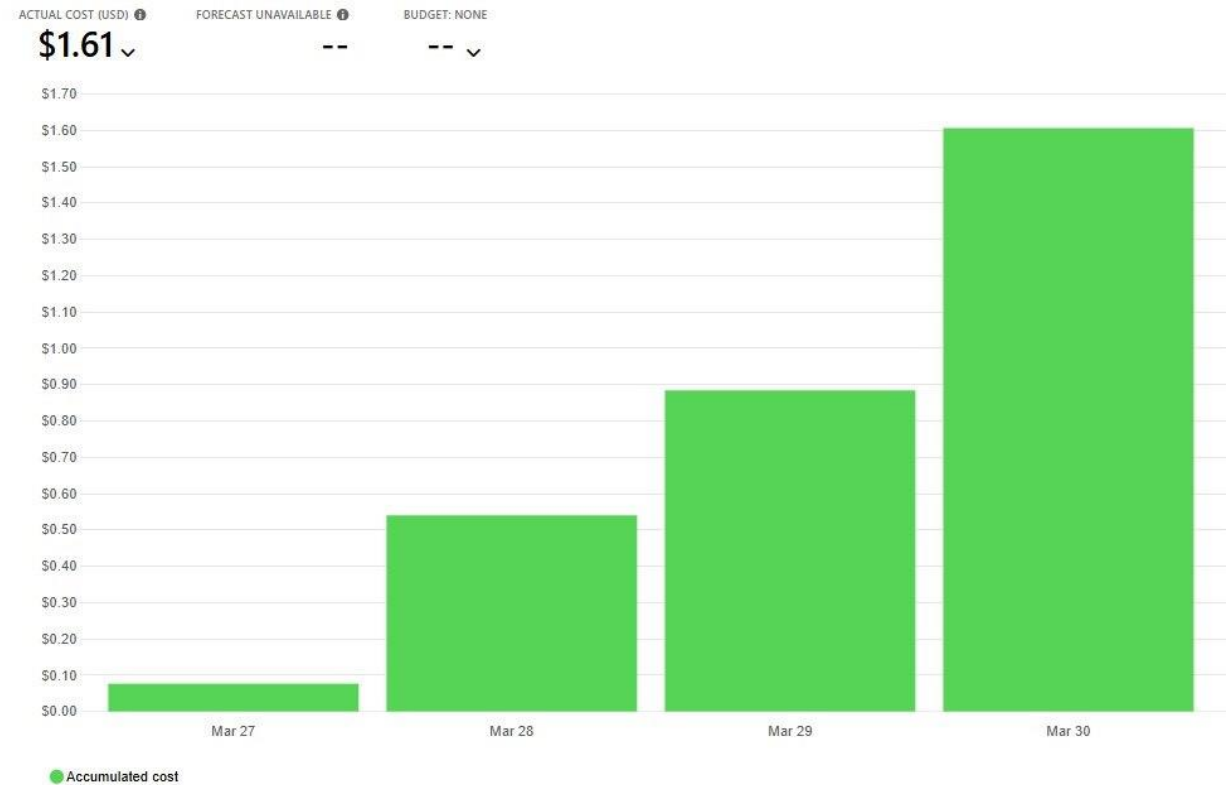
- When working with the system in an interactive mode,
 - Analyst chooses which PCAPs to analyse
 - Can override drill-down decisions
 - Single-day mode (--day) for targeted investigation
- Documentation for easy review
 - Checkpoint after every day — analyst can inspect intermediate results
 - Monitor log with timestamped progress updates
 - Report clearly labels confidence levels per finding (High / Medium / Low)
 - Evidence Gaps section explicitly lists what the agent could NOT determine
 - Assumptions and scope limitations stated per section



COST AND EFFICIENCY

MODEL, COST & CHART

- Model: Azure OpenAI GPT-4o-mini, Temperature 0
- ~180 LLM calls across all agents
- Data hosting (Google Drive + Apps Script API): free!
- Accumulated Azure cost: **\$1.61** over full run



TRADE-OFFS & EFFICIENCY

Runtime:

- Total: 6.2 hours
- Pass 1 (Zeek sweep): ~2.5 hours
- Pass 2 (PCAP drill-down): ~3.5 hours
- Report generation: ~5 minutes

Trade-offs:

- Zeek-only (no drill-down): ~2.5 hrs, medium accuracy, ~\$2
- Current (sweep + drill-down): ~6.2 hrs, high accuracy, ~\$3–6
- With local infrastructure (no API downloads): ~45 min

Hardware:

- Peak RAM: ~200 MB — logs streamed line-by-line
- CPU: ~6% single core — pipeline is network-bound not compute-bound
- No GPU required — all inference via Azure API



THANK YOU